



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de
Monterrey**

Evidencia 1 - Reflexión

Materia: Programación de estructuras de datos y algoritmos fundamentales (Gpo
607)

Profesor: David Alonso Cantú Delgado

Por:

Isis Krystal Agramon Leal - A00843802

20 de septiembre del 2026

Reflexión:

Trabajar con los archivos de logs me ayudó a entender mejor por qué es importante ordenar la información antes de buscar algo dentro de ella. Cuando los registros están desordenados, encontrar una fecha específica puede ser más complicado, especialmente cuando hay muchos datos. Al ordenarlos por fecha y hora, fue posible realizar búsquedas por rangos de una forma más organizada y utilizar búsqueda binaria para encontrar los límites sin tener que recorrer todos los registros.

Al comparar los algoritmos, pude notar que aunque todos llegan al mismo resultado de ordenar los registros, no todos lo hacen de la misma manera ni tardan lo mismo. Swap Sort, Bubble Sort, Selection Sort e Insertion Sort pueden llegar a tener una complejidad de $O(n^2)$, mientras que algoritmos como Merge Sort trabajan con $O(n \log n)$. Quick Sort también puede ser muy rápido, aunque su rendimiento depende de cómo se elija el pivote y de cómo estén acomodados los datos. Shell Sort, por otro lado, utiliza intervalos para ir acomodando los elementos antes de hacer las comparaciones más cercanas.

Los resultados mostraron claramente que el orden inicial de los datos puede cambiar mucho el rendimiento de algunos algoritmos. Por ejemplo, Insertion Sort tardó 4799.05 ms con el archivo desordenado, mientras que con el archivo casi ordenado tardó solamente 12.0827 ms. Algo parecido pasó con Bubble Sort, que pasó de 5263.75 ms a 14.848 ms. Esto me ayudó a entender que estos algoritmos pueden aprovechar bastante que los datos ya tengan cierto orden.

En Selection Sort ocurrió algo diferente, ya que los tiempos fueron prácticamente iguales: 505.436 ms con el primer archivo y 504.287 ms con el segundo. Esto tiene sentido porque el algoritmo sigue buscando el elemento menor entre los datos restantes aunque gran parte del archivo ya esté ordenado. Merge Sort también tuvo resultados bastante parecidos en ambos archivos, con 55.3929 ms y 49.1364 ms.

Un resultado que me llamó la atención fue el de Quick Sort. Con el archivo desordenado tardó 22.8603 ms, pero con el archivo casi ordenado aumentó a 152.107 ms. Al principio esperaba que el segundo archivo también fuera rápido por estar casi ordenado, pero al revisar cómo funciona mi implementación entendí que se utiliza el último elemento como pivote. Cuando los datos ya tienen cierto orden, este pivote puede generar divisiones menos equilibradas y hacer que Quick Sort realice más trabajo.

En sí, las mediciones coincidieron con gran parte de lo que esperaba. Los algoritmos como Bubble e Insertion mejoraron mucho con el archivo casi ordenado, mientras que Selection se mantuvo prácticamente igual. También pude ver que la complejidad teórica sirve para tener una idea del comportamiento de un algoritmo, pero el tiempo real también depende de cómo estén organizados los datos y de la forma en la que esté implementado.

Uso de inteligencia artificial:

Durante el desarrollo de la evidencia utilicé inteligencia artificial principalmente como apoyo para entender partes del código, revisar errores y adaptar funciones que ya había trabajado anteriormente. No utilicé las respuestas directamente sin revisarlas, ya que fui probando cada parte por separado para comprobar que realmente funcionara con los archivos de logs.

Algunos de los prompts más importantes que utilicé fueron preguntas sobre cómo adaptar mis algoritmos de ordenamiento anteriores para trabajar con objetos Log, cómo modificar una búsqueda binaria para encontrar el inicio y el final de un rango de fechas, y cómo medir el tiempo de ejecución de cada algoritmo utilizando chrono.

Uno de los errores que encontré durante el uso de IA ocurrió con Selection Sort. Durante la adaptación y revisión del algoritmo, el intercambio de elementos quedó colocado dentro del ciclo que busca el elemento menor. Al principio parecía que funcionaba porque la prueba se había realizado después de otro algoritmo que ya había ordenado el vector. Para comprobarlo mejor, probé Selection Sort de manera independiente y agregué una verificación que recorría la lista para revisar si realmente estaba ordenada. El resultado indicó que no lo estaba. Después revisé nuevamente el algoritmo y moví el intercambio fuera del ciclo interno, para que primero encontrara el elemento menor y después realizara un solo intercambio. Al volver a ejecutar la prueba, la lista sí quedó ordenada.

Modificaciones y aprendizaje:

A partir de las sugerencias de la IA hice diferentes modificaciones para adaptarlas a lo que necesitaba mi programa. Por ejemplo, utilicé la key de cada objeto Log para realizar las comparaciones entre fechas y adapté la búsqueda binaria para encontrar el primer registro mayor o igual a la fecha inicial y el primer registro mayor a la fecha final. Esto también permitió incluir correctamente los registros que tenían la misma fecha y hora sin perder duplicados. Además, fui haciendo pruebas después de los cambios para asegurarme de que los resultados fueran correctos.

Me ayudó a entender mejor que no basta con que un programa compile o muestre resultados que parecen correctos. También es necesario probar diferentes casos y entender qué está haciendo cada algoritmo. El error de Selection Sort fue un ejemplo de esto, porque inicialmente parecía funcionar, pero al probarlo de manera independiente pude detectar que realmente no estaba ordenando correctamente. También entendí mejor cómo el estado inicial de los datos puede afectar bastante el rendimiento de algunos algoritmos.

En conclusión, esta evidencia me permitió relacionar los algoritmos de ordenamiento y búsqueda con un caso más cercano a una aplicación real. Antes los había trabajado principalmente con listas de números, pero al utilizarlos con registros de logs pude entender mejor para qué pueden servir. También pude comparar su rendimiento con datos diferentes y comprobar que elegir un algoritmo no depende solamente de su complejidad teórica, sino también de los datos y de cómo esté implementado. Además, el proceso de probar y corregir el programa me ayudó a comprender mejor mi propio código y a no asumir que una solución es correcta solamente porque compila.